DL Practicals – What Changes in Each Code

Practicals 1 to 5 · learn the base code, then only the changed lines

Practical 1 – FNN (Feedforward Neural Network)

Learn 1a fully. The others are 1a with a few lines changed.

Base code = 1a (Handwritten MNIST)

```
a. import pandas, matplotlib, keras
b. read train csv + test csv  → x = pixels / 255, y = "label" column
c. Dense(128, relu) → Dense(64, relu) → Dense(10, softmax)      input_shape=(784,)
d. compile(optimizer="sgd", loss="sparse_categorical_crossentropy") → fit(epochs=11)
e. evaluate → print loss, accuracy
f. plot loss and accuracy
```

What changes in each code

Line	1a MNIST	1b Fashion MNIST	1c CIFAR10	1e Sign MNIST
CSV files	`mnist_train.csv`, `mnist_test.csv`	`fashion-mnist_train.csv`, `fashion-mnist_test.csv`	`cifar-10.csv` (only one file)	`sign_mnist_train.csv`, `sign_mnist_test.csv`
Train/test split	separate files	separate files	**`train_test_split(test_size=0.2)`**	separate files
input_shape	784	784	**3072** (32×32×3)	784
Hidden layers	128 → 64	128 → 64	**256 → 128**	128 → 64
Output units	10	10	10	**25**
Optimizer	sgd	sgd	**adam**	**adam**
Epochs	**11**	10	10	10

Exact changes

1b (vs 1a) – only file names and epochs:

```
train = pd.read_csv("fashion-mnist_train.csv")
test = pd.read_csv("fashion-mnist_test.csv")
...
h = model.fit(x_train, y_train, epochs=10, ...)
```

1c (vs 1a) – one CSV, so loading part is different:

```
from sklearn.model_selection import train_test_split     # extra import

data = pd.read_csv("cifar-10.csv")
x = data.drop("label", axis=1).values / 255.0
y = data["label"].values
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

and the model:

```
keras.layers.Dense(256, activation="relu", input_shape=(3072,)),
keras.layers.Dense(128, activation="relu"),
...
model.compile(optimizer="adam", ...)
```

1e (vs 1a) – file names, 25 outputs, adam:

```
train = pd.read_csv("sign_mnist_train.csv")
test = pd.read_csv("sign_mnist_test.csv")
...
keras.layers.Dense(25, activation="softmax")    # labels 0-24
...
model.compile(optimizer="adam", ...)
```

Practical 2 – CNN (Convolutional Neural Network)

Learn 2b fully. 2c is 2b with 3 lines changed. 2a has a different loading part.

CNN = same as Practical 1 FNN, plus:

1. **reshape** pixels into images
2. **Conv2D + MaxPooling2D** layers before Flatten and Dense

Base code = 2b (Handwritten MNIST)

```
a. read train csv + test csv → x = pixels / 255, y = "label"
   x = x.reshape(-1, 28, 28, 1)                    ← image shape
b. Conv2D(32) → MaxPool → Conv2D(64) → MaxPool → Flatten → Dense(128) → Dense(10, softmax)
c. compile(adam, sparse_categorical_crossentropy) → fit(epochs=5)
d. evaluate → print loss, accuracy → plot
```

What changes in each code

Line	2a CIFAR10	2b MNIST	2c Sign MNIST
CSV files	`cifar-10.csv` (one file)	`mnist_train.csv`, `mnist_test.csv`	`sign_mnist_train.csv`, `sign_mnist_test.csv`
Train/test split	**`train_test_split(test_size=0.2)`**	separate files	separate files
Reshape	**`reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1)`**	`reshape(-1, 28, 28, 1)`	`reshape(-1, 28, 28, 1)`
input_shape	**(32, 32, 3)**	(28, 28, 1)	(28, 28, 1)
Output units	10	10	**25**
Epochs	10	5	5

Exact changes

2c (vs 2b) – only file names and output units:

```
train = pd.read_csv("sign_mnist_train.csv")
test = pd.read_csv("sign_mnist_test.csv")
...
keras.layers.Dense(25, activation="softmax")    # labels 0-24
```

2a (vs 2b) – one colour CSV:

```
from sklearn.model_selection import train_test_split     # extra import

data = pd.read_csv("cifar-10.csv")
x = data.drop("label", axis=1).values / 255.0
y = data["label"].values
x = x.reshape(-1, 3, 32, 32).transpose(0, 2, 3, 1)       # colour image 32x32x3
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
...
keras.layers.Conv2D(32, (3, 3), activation="relu", input_shape=(32, 32, 3)),
...
h = model.fit(x_train, y_train, epochs=10, ...)
```

Practical 3 – Autoencoder (Anomaly Detection)

All 5 codes have the SAME 5 steps and the SAME variable names (`x`, `y`, `x_normal`).

```
1. LOAD + SCALE      → x = data, y = labels (0 normal, 1 anomaly)
2. NORMAL DATA ONLY  → x_normal = x[y == 0]
3. MODEL             → Dense / LSTM / Conv autoencoder, compile(adam, mse, mae)
4. TRAIN             → model.fit(x_normal, x_normal)        ← input = output
5. DETECT            → the 4 lines below (same in every code)
```

Step 5 – copy-paste in every code

```
error = np.mean((x - model.predict(x)) ** 2, axis=1)       # reconstruction error
threshold = np.percentile(error[y == 0], 95)               # 95% of normal data is below this
y_pred = (error > threshold).astype(int)                   # above threshold → anomaly
print(classification_report(y, y_pred))
```

Only `axis` changes: **1** for rows (3a, 3f) · **(1, 2)** for sequences (3c, 3d) · **(1, 2, 3)** for images (3b).

Only 3 model types to remember

Model	Used in	Layers
Dense	3a, 3f	`Dense(32) → Dense(16) → Dense(32) → Dense(n)`
LSTM	3c, 3d	`LSTM(32) → RepeatVector(10) → LSTM(32, return_sequences=True) → TimeDistributed(Dense(n))`
Conv	3b	`Conv2D(32) → MaxPooling2D → Conv2D(32) → UpSampling2D → Conv2D(3, sigmoid)`

3a and 3f have the exact same model. 3c and 3d have the exact same model.

Only Step 1 (loading) is different

	Data file	x	y
3a	`creditcard.csv`	all columns except `Class`, MinMaxScaler	`Class`
3c	`energy consumption.csv`	`ffill()`, drop `Timestamp`, `Anomaly_Label`, MinMaxScaler, windows	`Anomaly_Label`
3d	`NASA TurboFan Engine.csv`	drop `unit`, `cycle`, `RUL`, `Fault_Label`, MinMaxScaler, windows	`Fault_Label`
3f	`sign_mnist_train/test.csv`	test pixels / 255	`label == 24` (Y)
3b	`bottle/` image folders	test images resized 128×128 / 255	0 if folder is `good` else 1

Exact Step 1 code

3a

```
data = pd.read_csv("creditcard.csv")
x = data.drop("Class", axis=1).values
y = data["Class"].values
x = MinMaxScaler().fit_transform(x)
x_normal = x[y == 0]
```

3c (3d is the same – only file name, dropped columns and label name change, and no `ffill`)

```
data = pd.read_csv("energy consumption.csv")
data = data.ffill()
features = data.drop(["Timestamp", "Anomaly_Label"], axis=1).values
labels = data["Anomaly_Label"].values
features = MinMaxScaler().fit_transform(features)

x, y = [], []                                  # windows of 10 rows for LSTM
for i in range(len(features) - 10):
    x.append(features[i:i + 10])
    y.append(labels[i + 9])
x, y = np.array(x), np.array(y)
x_normal = x[y == 0]
```

3d – change only these lines in 3c:

```
data = pd.read_csv("NASA TurboFan Engine.csv")
features = data.drop(["unit", "cycle", "RUL", "Fault_Label"], axis=1).values
labels = data["Fault_Label"].values
```

3f – train on letters except Y, detect on test file:

```
x_train = train.drop("label", axis=1).values / 255.0
x = test.drop("label", axis=1).values / 255.0
y = (test["label"].values == 24).astype(int)
x_normal = x_train[train["label"].values != 24]
```

3b – images:

```
test_paths = glob.glob("bottle/test/*/*.png")
x = load_images(test_paths)
y = np.array([0 if "/good/" in p else 1 for p in test_paths])
x_normal = load_images(glob.glob("bottle/train/good/*.png"))
```

Practical 4 – CBOW (Word2Vec)

All 4 codes start with the SAME 3 steps. Only the LOAD lines and the last part change.

```
1. LOAD      → texts = list of strings            (different in each code)
2. CLEAN     → sentences = [clean(t) for t in texts]   (same in all)
3. TRAIN     → w2v = Word2Vec(..., sg=0)               (same in all)
4. USE       → 4b, 4c: show similar words  |  4a, 4d: doc_vector + classify / recommend
```

Steps 2 + 3 – same in every code

```
def clean(text):
    text = re.sub("[^a-z ]", " ", text.lower())                      # lowercase, keep letters
    return [w for w in text.split() if w not in ENGLISH_STOP_WORDS]   # tokenize + remove stop words

sentences = [clean(t) for t in texts]

w2v = Word2Vec(sentences, vector_size=100, window=5, min_count=1, sg=0, epochs=10)   # sg=0 → CBOW
```

Only 4c uses `epochs=200` (tiny document needs more training).

Step 1 – LOAD (the only part you must remember per code)

	Load lines
4a	`data = pd.read_csv("IMDB Dataset.csv")` · `texts = data["review"].tolist()` · `y = (data["sentiment"] == "positive").astype(int).values`
4b	`data = pd.read_csv("twitter_dataset.csv")` · `texts = data["Text"].tolist()`
4c	`texts = open("document3.txt", encoding="utf-8").read().split(".")`
4d	`data = pd.read_csv("twitter_dataset.csv")` · `texts = data["Text"].tolist()`

Step 4 – two kinds of ending

4b and 4c are IDENTICAL here – show embeddings and similar words:

```
words = w2v.wv.index_to_key[:5]                      # 5 most common words
print("Embedding of", words[0], ":", w2v.wv[words[0]][:10])
for w in words:
    print("Most similar to", w, ":", w2v.wv.most_similar(w, topn=5))
print("Similarity", words[0], words[1], ":", w2v.wv.similarity(words[0], words[1]))
```

4a and 4d share the SAME doc_vector (document = average of its word vectors):

```
def doc_vector(words):
    words = [w for w in words if w in w2v.wv]
    return np.mean([w2v.wv[w] for w in words], axis=0) if words else np.zeros(100)

x = np.array([doc_vector(s) for s in sentences])
```

then **4a** classifies:

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
model = keras.Sequential([
    keras.layers.Dense(64, activation="relu", input_shape=(100,)),
    keras.layers.Dense(1, activation="sigmoid")              # positive / negative
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(x_train, y_train, epochs=10)
loss, acc = model.evaluate(x_test, y_test)
```

and **4d** recommends:

```
query = doc_vector(clean(input("Enter keywords: ")))
scores = cosine_similarity([query], x)[0]
for i in scores.argsort()[::-1][:5]:                  # top 5, highest first
    print(round(scores[i], 3), "->", texts[i][:100])
```

Practical 5 – Transfer Learning (VGG16)

All 3 codes are the SAME. Only ONE line changes: `DATA = "..."`. Learn 5a only.

Base code = 5a

```
a. import keras, VGG16, preprocess_input
b. image_dataset_from_directory(DATA, validation_split=0.2) → train_ds, test_ds   (images resized to
64x64)
c. base = VGG16(weights=WEIGHTS, include_top=False)
d. base.trainable = False
   Lambda(preprocess_input) → base → Flatten → Dense(256) → Dropout(0.5) → Dense(classes, softmax)
e. fit(epochs=5)
f/g. evaluate → loss, accuracy
h. show 9 test images with True (T) and Predicted (P) labels
i. fine tune: unfreeze last 4 VGG layers → compile(Adam(1e-5)) → fit(epochs=3) → evaluate
```

The only change

	Line 9
5a	`DATA = "caltech-101"`
5b	`DATA = "intel_images"`
5c	`DATA = "fashion_mnist"`

Why the same code works for all 3

- **SIZE = 64** for every dataset – VGG16 needs at least 32×32, and 64 is fast.
- `image_dataset_from_directory` loads images as **RGB by default**, so grayscale Fashion MNIST automatically becomes 3 channels.
- Number of classes is never typed – it is `len(class_names)` (number of folders): 102 / 6 / 10.